Run Screenshots

End-to-End Data Management Pipeline for a Book Recommendation System

Data Management for Machine Learning - Assignment 2

Name: Karthik S Rajan Roll No.: 2025AG05779

GitHub repository: https://github.com/karthiksrajan/recommendation-pipeline

Screenshots taken from the executed Kaggle notebook (Run All, no errors). The last page is the run graph generated by the pipeline itself (reports/orchestration_run.png).

Contents

Page 2 Figure 1 - Validation findings per rule
Page 3 Figure 2 - Time-based split and derived datasets
Page 4 Figure 3 - MLflow experiment tracking
Page 5 Figure 4 - Model comparison table (test window, K = 5 and 10)
Page 6 Figure 5 - Result figures
Page 7 Figure 6 - Prediction-time recommendations
Page 8 Figure 7 - Task registry and dependencies
Page 9 Figure 8 - Successful orchestrated run
Page 10 Figure 9 - Task table of the successful run
Page 11 Figure 10 - Scheduled re-run (idempotence)
Page 12 Figure 11 - Negative test: the data-quality gate stops a bad run
Page 13 Figure 12 - Reproducibility check
Page 14 Figure 13 - Deliverables check
Page 15 Figure 14 - Headline results and key inferences
Page 16 Generated figure - successful run (reports/orchestration_run.png)

Figure 1 - Validation findings per rule

Rejected rules (red) and repaired / warning rules (yellow) across the users, items and interactions tables, as produced by the Pandera and custom rule checks.

```
Notes: (1) 'orphan user_id' detected > injected because interactions of users rejected for an invalid signup_date also become orphans (cascade).
       (2) 'timestamp before signup' can be 1 lower because a row whose user was rejected has no signup date to compare with – it is rejected as an orphan instead.
```

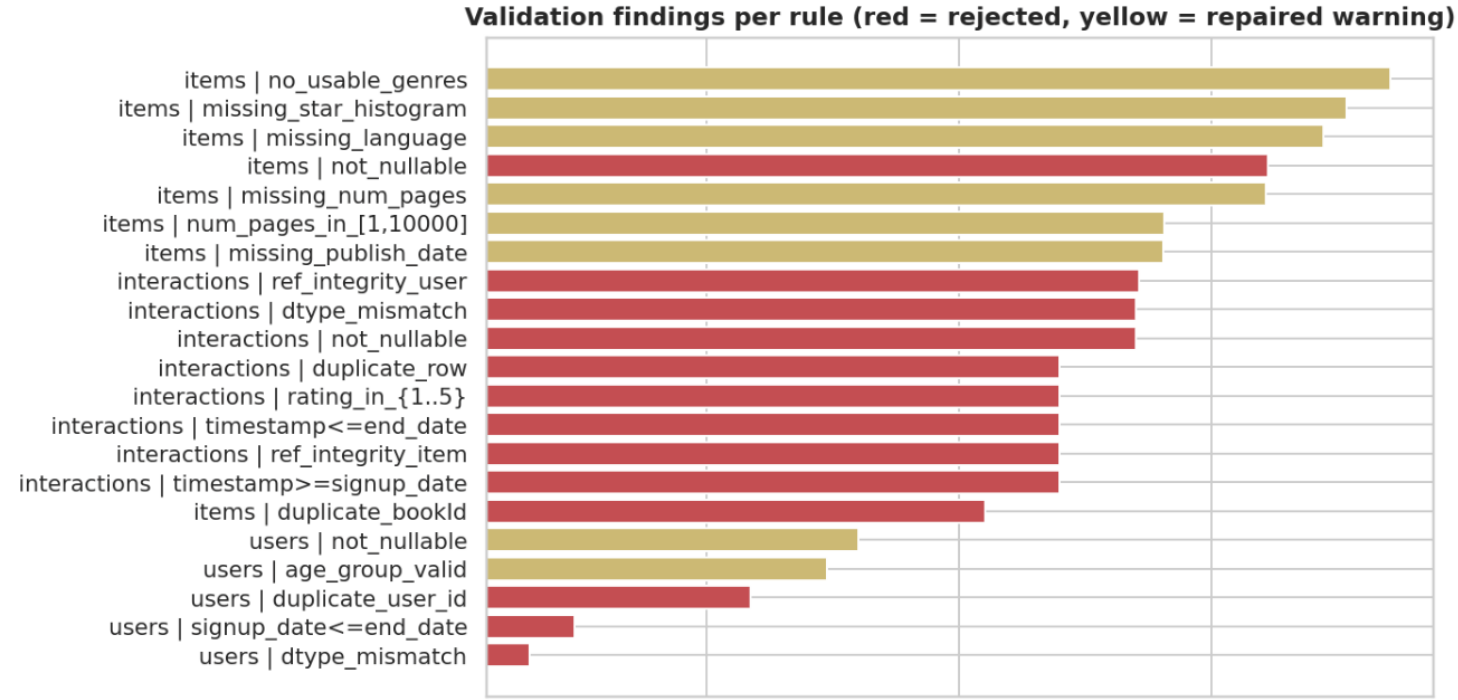
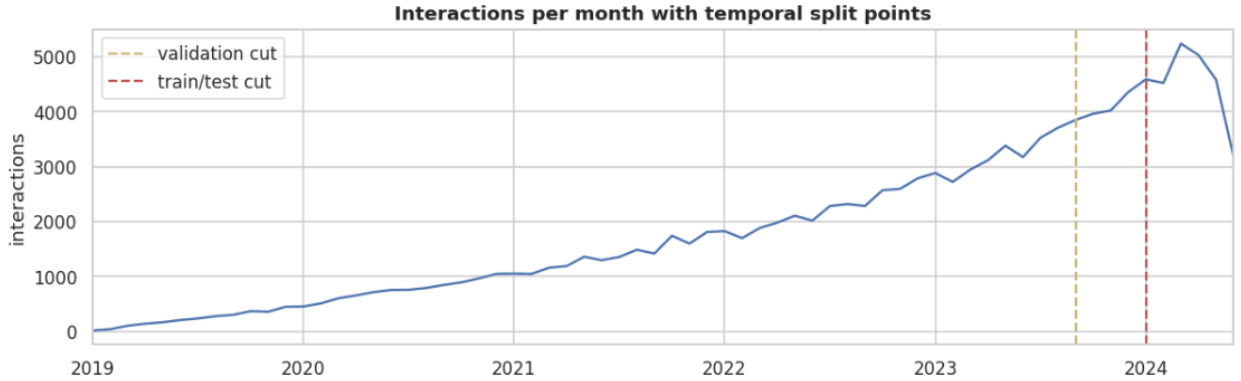


Figure 2 - Time-based split and derived datasets

Monthly interaction volume with the validation cut and the train/test cut marked, followed by the user-activity summary and the item-popularity table.

```
[recsys.transformation] train 98,667 | test 24,667 | (train_fit 83,867 / val 14,800)
[recsys.transformation] test users never seen in train (cold-start): 179 of 3401 | test items unseen in train: 1 of 2907
[recsys.transformation] user-item matrix: 3,816 users x 2,999 items | nnz=98,667 | sparsity=99.14%
```



	user_id	n_interactions	avg_rating	rating_std	n_positive	n_categories	first_interaction	last_interaction	positive_ratio	active_span_days	activity_segment
0	U00001	67	4.417910	0.872983	56	18	2020-12-10 22:07:34	2023-12-31 20:41:07	0.835821	1115	power
1	U00002	16	4.312500	0.873212	14	5	2020-11-23 05:40:52	2023-09-20 13:16:32	0.875000	1031	medium
2	U00003	19	4.789474	0.535303	18	10	2020-06-02 16:06:12	2023-09-30 15:24:58	0.947368	1214	medium

	item_id	title	category	n_interactions	avg_rating	bayes_avg_rating	popularity_score
0	3	Harry Potter and the Sorcerer's Stone	Fantasy	367	4.640327	4.623065	1.000000
1	3356	Harry Potter y la piedra filosofal	Fantasy	326	4.610429	4.592850	0.980007
2	4255	Harry Potter and the Philosopher's Stone	Fantasy	315	4.603175	4.585451	0.974215
3	15871	Harry Potter à l'école des sorciers	Fantasy	304	4.648026	4.626932	0.968218
4	15868	Harry Potter y la piedra filosofal	Fantasy	284	4.633803	4.612257	0.956738

Figure 3 - MLflow experiment tracking

Top-level MLflow run for each model, with every tuning trial stored as a nested run underneath.

```
print("To browse the MLflow UI locally:  mlflow ui --backend-store-uri sqlite:///mlflow.db")
```

```
36 runs in experiment 'goodreads-recsys' (8 top-level, rest = nested tuning trials)
```

	runName	model	ndcg_at_10	recall_at_10	fit_seconds	pipeline_mode
0	evaluation_summary	None	NaN	NaN	NaN	interactive
1	train_als	ALS (implicit MF)	0.036478	0.045762	0.663267	interactive
2	train_content	Content-based (genres)	0.011226	0.013503	0.007402	interactive
3	train_hybrid	Hybrid (ItemKNN + ALS + content)	0.044159	0.055041	1.175520	interactive
4	train_itemknn	ItemKNN (cosine CF)	0.044903	0.056669	0.522040	interactive
5	train_popularity	Popularity (baseline)	0.026609	0.032443	0.001376	interactive
6	train_random	Random (sanity floor)	0.002846	0.003980	0.000288	interactive
7	train_svd	SVD (Surprise MF)	0.001499	0.002381	1.087422	interactive

```
To browse the MLflow UI locally:  mlflow ui --backend-store-uri sqlite:///mlflow.db
```

(i) Serving & Reporting layer

i.1 Prediction-time recommendations (with cold-start fallback)

`src/serving/recommend.py` turns the best personalised model into top-K lists. Prediction-time user context comes from the **same** `FeatureStore` used for training; users without any training history (cold start) receive a **popularity fallback** instead of an empty or random list. The pipeline's batch task writes top-10 lists for every user to `reports/top10_recommendations.parquet` .

[38]:

```
%%writefile src/serving/recommend.py
"""Prediction-time layer: top-K recommendations from the best personalised model, features from the SAME FeatureStore,
and a popularity fallback for cold-start users (users with no training history)."""
from __future__ import annotations

import json
```

Figure 4 - Model comparison table (test window, K = 5 and 10)

Precision, Recall, NDCG, Coverage, Diversity, RMSE and MAE for all seven models, with the change relative to the popularity baseline and the bootstrap confidence intervals.

```
[recsys.evaluation] results table -> reports/model_results.csv | best personalised model by NDCG@10: ItemKNN (cosine CF) (0.0449)
{'best_model': 'ItemKNN (cosine CF)', 'best_NDCG@10': 0.044903219482368105, 'n_models': 7}
```

Test-window results (K = [5, 10]; green = best in column)

Model	Precision@5	Recall@5	NDCG@5	Precision@10	Recall@10	NDCG@10	Coverage@10	Diversity@10	RMSE	MAE
Random (sanity floor)	0.0016	0.0020	0.0020	0.0017	0.0040	0.0028	1.0000	0.8009	n/a	n/a
Popularity (baseline)	0.0197	0.0201	0.0242	0.0154	0.0324	0.0266	0.0080	0.2960	0.9336	0.7540
ALS (implicit MF)	0.0242	0.0248	0.0307	0.0214	0.0458	0.0365	0.2644	0.5886	n/a	n/a
SVD (Surprise MF)	0.0008	0.0010	0.0009	0.0011	0.0024	0.0015	0.0263	0.8150	0.9317	0.7316
ItemKNN (cosine CF)	0.0295	0.0300	0.0377	0.0260	0.0567	0.0449	0.1907	0.4050	n/a	n/a
Content-based (genres)	0.0086	0.0091	0.0102	0.0069	0.0135	0.0112	0.7312	0.1168	n/a	n/a
Hybrid (ItemKNN + ALS + content)	0.0303	0.0322	0.0377	0.0265	0.0550	0.0442	0.1871	0.4888	n/a	n/a

Relative lift vs. popularity baseline

Model	Precision@5	Recall@5	NDCG@5	Precision@10	Recall@10	NDCG@10	Coverage@10	Diversity@10
Random (sanity floor)	-92.0%	-90.3%	-91.6%	-89.0%	-87.7%	-89.3%	+12395.8%	+170.5%
ALS (implicit MF)	+23.3%	+23.5%	+26.5%	+39.3%	+41.1%	+37.1%	+3204.2%	+98.8%
SVD (Surprise MF)	-95.7%	-94.8%	-96.3%	-92.6%	-92.7%	-94.4%	+229.2%	+175.3%
ItemKNN (cosine CF)	+50.2%	+49.1%	+55.6%	+69.0%	+74.7%	+68.8%	+2283.3%	+36.8%
Content-based (genres)	-56.1%	-54.8%	-58.0%	-55.4%	-58.4%	-57.8%	+9037.5%	-60.6%
Hybrid (ItemKNN + ALS + content)	+53.8%	+60.5%	+55.6%	+72.0%	+69.7%	+66.0%	+2237.5%	+65.1%

[33]:
```
ci = pd.read_csv(BASE / "reports/model_results_ci.csv")
display(ci.pivot(index="Model", columns="metric", values="mean").join(
        ci[ci.metric == f"NDCG@{K}"].set_index("Model")[["ci_low", "ci_high"]].add_prefix(f"NDCG@{K} ")).round(4).loc[res_df.index]
        .style.set_caption("Mean per-user metrics with 95% bootstrap CI for NDCG"))
```

Figure 5 - Result figures

Bootstrap confidence intervals for Precision, Recall and NDCG at K = 10, together with coverage / diversity of the recommendation lists.

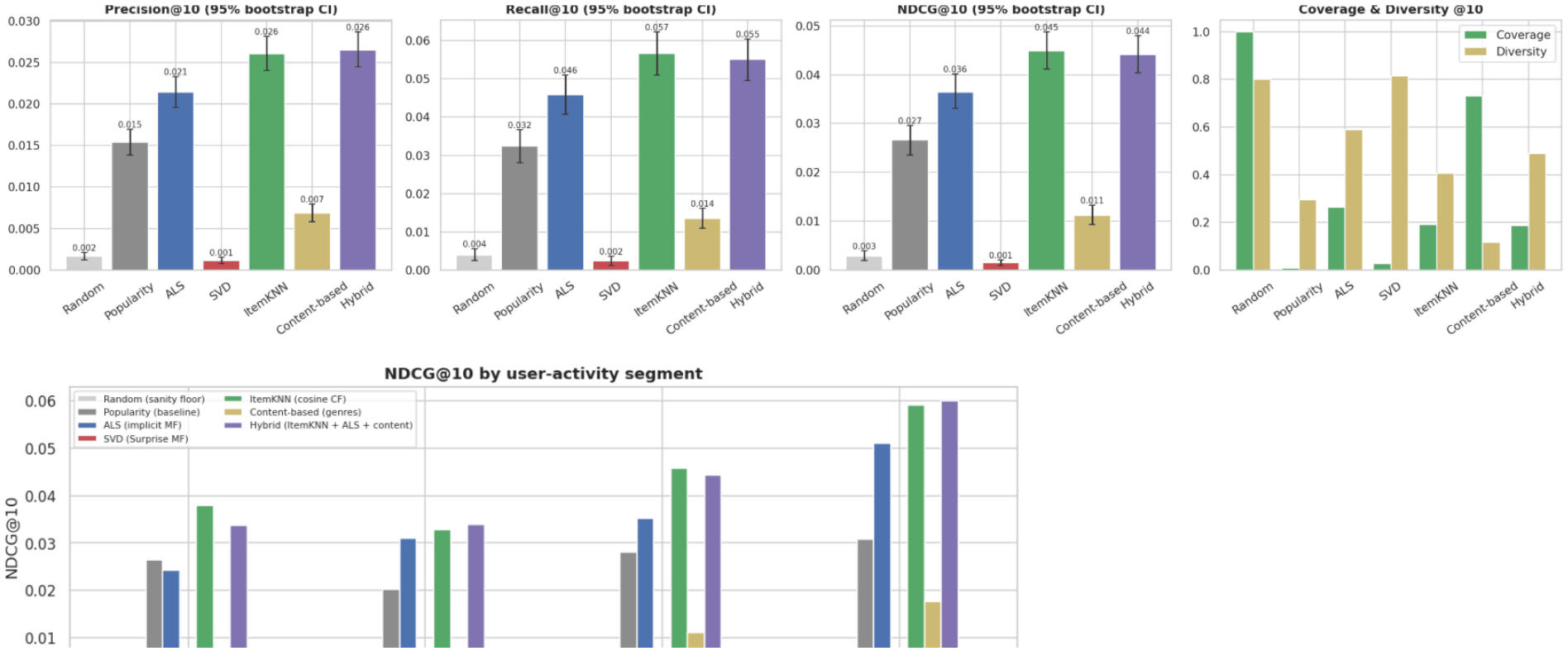


Figure 6 - Prediction-time recommendations

Top-10 list for a known user from the best model (ItemKNN) and the popularity fallback for a cold-start user.

[39]:
```
recommend = load("src.serving.recommend")
rec_engine = recommend.Recommender(BASE)
print("serving model:", rec_engine.model_name)
print("known user  :", rec_engine.user_context(sample_user)); display(rec_engine.recommend(sample_user, 5)[["rank", "title", "category", "source"]])
cold_user = sorted(set(test.user_id) - set(train.user_id))[0]
print("cold-start  :", rec_engine.user_context(cold_user)); display(rec_engine.recommend(cold_user, 5)[["rank", "title", "category", "source"]])
```

```
serving model: ItemKNN (cosine CF)
known user  : {'user_id': 'U03672', 'cold_start': False, 'preferred_category': 'Christian', 'activity_count': 22, 'avg_rating': 4.27, 'segment': 'active'}
```

	rank	title	category	source
0	1	The Hobbit	Fantasy	ItemKNN (cosine CF)
1	2	The Kite Runner	Historical	ItemKNN (cosine CF)
2	3	Freakonomics: Un economista políticamente incorrecto explora el lado oculto ...	Economics	ItemKNN (cosine CF)
3	4	Harry Potter y la piedra filosofal	Fantasy	ItemKNN (cosine CF)
4	5	The Hobbit	Fantasy	ItemKNN (cosine CF)

```
cold-start  : {'user_id': 'U00011', 'cold_start': True}
```

	rank	title	category	source
0	1	Harry Potter and the Sorcerer's Stone	Fantasy	popularity fallback (cold start)
1	2	Harry Potter y la piedra filosofal	Fantasy	popularity fallback (cold start)
2	3	Harry Potter à l'école des sorciers	Fantasy	popularity fallback (cold start)
3	4	Harry Potter and the Philosopher's Stone	Fantasy	popularity fallback (cold start)
4	5	Harry Potter y la piedra filosofal	Fantasy	popularity fallback (cold start)

i.2 Reporting

`src/reporting/final_report.py` assembles the **2–3 page final report (PDF)** — design decisions, tools, validation results, features, results table and chart, orchestration, limitations — *directly from the artefacts of the latest run*, so the report can never disagree with the numbers. `publish.py` is the pipeline's last task: it regenerates figures, the run manifest and the PDF.

Figure 7 - Task registry and dependencies

The 16 workflow tasks, their dependency levels and the tasks each one depends on.

```
compile((BASE / "dags/recommendation_pipeline_dag.py").read_text(), "recommendation_pipeline_dag.py", "exec")      # Airflow is not installed here: syntax check only
print("Airflow DAG file written to dags/recommendation_pipeline_dag.py (syntax OK; shares the task registry with the Python engine)\n")

wf = tasks.build_workflow(BASE, CFG)
display(pd.DataFrame(wf.dry_run(), columns=["task", "level", "depends_on"]).assign(description=[wf.tasks[t].description for t in wf.topological_order()]))
```

```
Airflow DAG file written to dags/recommendation_pipeline_dag.py (syntax OK; shares the task registry with the Python engine)
```

	task	level	depends_on	description
0	ingest_data	0	[]	load Goodreads items, simulate users + interactions, inject faults
1	validate_data	1	[ingest_data]	Pandera schemas + custom rules -> error dataset + validation report
2	quality_gate	2	[validate_data]	stop the pipeline when too many rows are rejected
3	prepare_data	3	[quality_gate]	cleaning, imputation, standardisation
4	transform_split	4	[prepare_data]	time-based split + activity / popularity / matrix
5	build_features	5	[transform_split]	user / item / interaction features + catalog
6	train_random	6	[build_features]	random sanity floor
7	train_popularity	6	[build_features]	popularity baseline
8	train_als	6	[build_features]	implicit ALS (tuned)
9	train_svd	6	[build_features]	Surprise SVD (tuned)
10	train_itemknn	6	[build_features]	item-based CF (tuned)
11	train_content	6	[build_features]	content-based genre profile
12	train_hybrid	7	[train_als, train_itemknn, train_content]	hybrid blend (tuned weights)
13	evaluate_models	8	[train_random, train_popularity, train_als, train_svd, train_itemknn, train_...	results table, bootstrap CIs, significance tests, MLflow summary
14	batch_recommend	9	[evaluate_models]	top-10 lists for every user (prediction-time features)
15	publish_reports	10	[evaluate_models, batch_recommend]	figures + final PDF report + run manifest

i1 Run the complete workflow (all 16 tasks, from raw data to PDF report)

Figure 8 - Successful orchestrated run

DAG of the full run: all 16 tasks succeeded in 63.4 s, shown with the per-task status and duration.

```
[recsys.serving] batch recommendations (ItemKNN (cosine CF)): 3,816 users x 10 -> reports/top10_recommendations.parquet
[recsys.orchestrator] > publish_reports (attempt 1)
[recsys.reporting] figures, run manifest and final report written -> reports/final_report.pdf
[recsys.orchestrator] === run 20261112-044122 finished: success in 63.4s ===
```

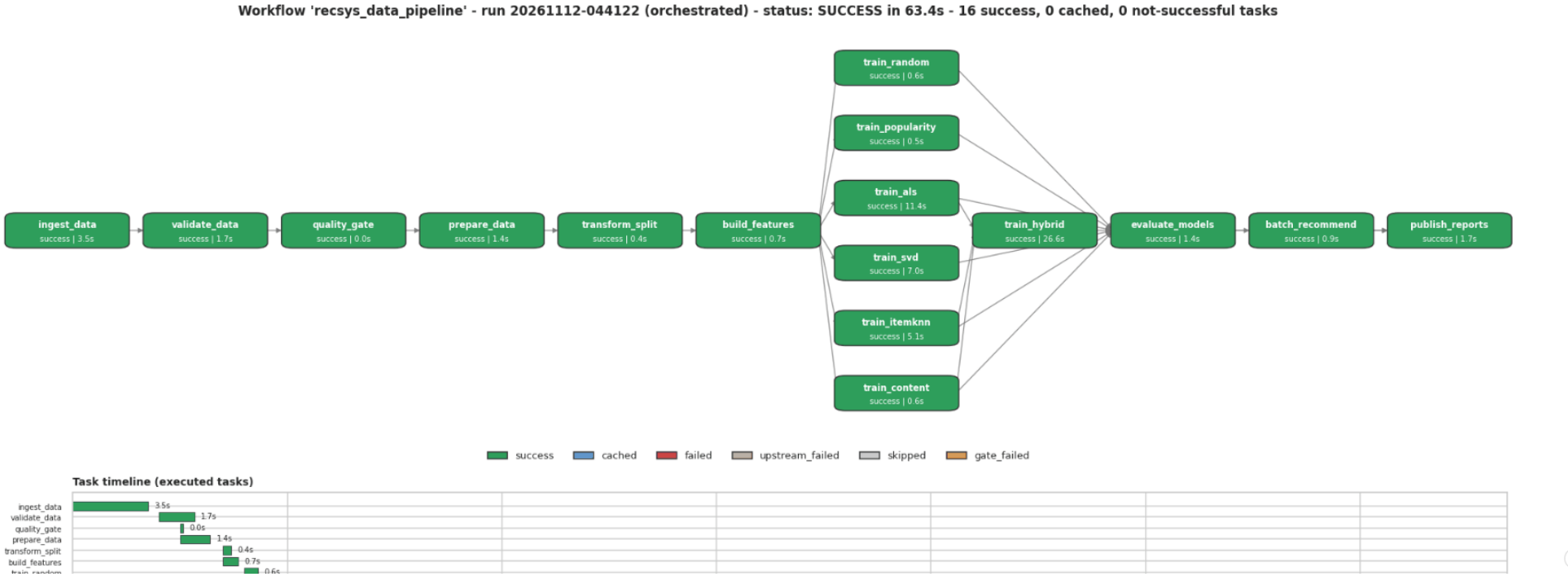
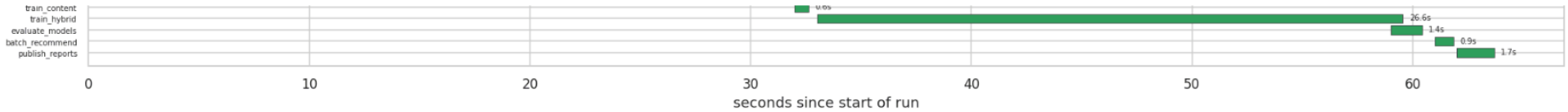


Figure 9 - Task table of the successful run

Timeline of the executed tasks and the task table (status, attempts, seconds) for the same run.



	name	status	attempts	seconds	message
0	ingest_data	success	1	3.50	
1	validate_data	success	1	1.67	
2	quality_gate	success	1	0.00	
3	prepare_data	success	1	1.40	
4	transform_split	success	1	0.37	
5	build_features	success	1	0.71	
6	train_random	success	1	0.61	
7	train_popularity	success	1	0.53	
8	train_als	success	1	11.42	
9	train_svd	success	1	6.97	
10	train_itemknn	success	1	5.10	
11	train_content	success	1	0.61	
12	train_hybrid	success	1	26.55	
13	evaluate_models	success	1	1.42	
14	batch_recommend	success	1	0.86	
15	publish_reports	success	1	1.69	

j.2 Reproducibility check — orchestrated run vs. stage-by-stage run

Figure 10 - Scheduled re-run (idempotence)

Second run with resume enabled: all 16 tasks are served from cache and nothing is recomputed.

```
orch.render_run_graph(rec2, BASE / "reports/orchestration_resume_run.png"); plt.show()
print(f"{sum(t['status'] == 'cached' for t in rec2['tasks'])} of {len(rec2['tasks'])} tasks served from cache in {rec2['seconds']}s (first run: {rec['seconds']}s)")
```

```
[recsys.orchestrator] === run 20261112-044227 | workflow 'recsys_data_pipeline' | 16 tasks | resume=True ===
[recsys.orchestrator] === run 20261112-044227 finished: success in 0.0s ===
```

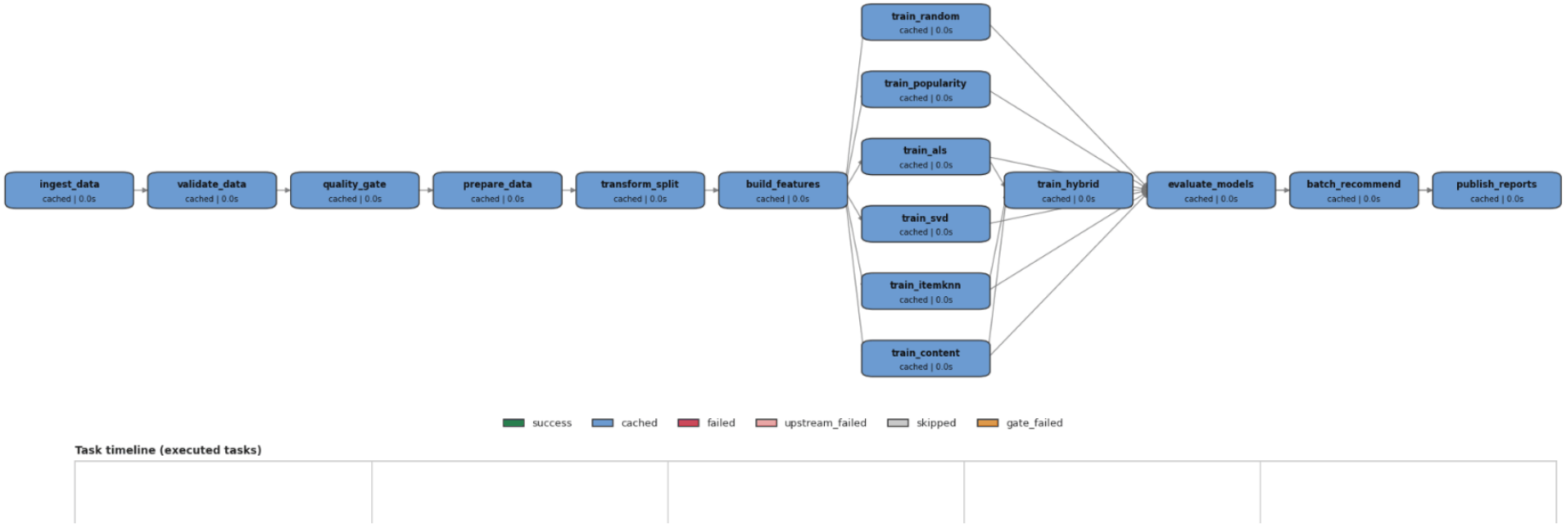


Figure 11 - Negative test: the data-quality gate stops a bad run

With the rejection threshold lowered to 0.1 %, the quality gate fails and every downstream task is skipped.

```
[recsys.ingestion] items loaded: (20068, 23) | boilerplate genre prefix in 100.0% of genre strings
[recsys.ingestion] source extracts written to data/raw/: items=(20068, 23) users=(4015, 4) interactions=(126106, 5)
[recsys.orchestrator] > validate_data (attempt 1)
[recsys.validation] users         rows=  4,015 | rejected=    20 | warning rows=   69
[recsys.validation] items         rows= 20,068 | rejected=   684 | warning rows=7,110
[recsys.validation] interactions  rows=126,106 | rejected= 2,772 | warning rows=    0
[recsys.validation] interactions rejected: 2,772 (2.20%) | report -> reports/data_validation_report.html
[recsys.orchestrator] > quality_gate (attempt 1)
[recsys.orchestrator] QUALITY GATE FAILED in quality_gate: interaction reject ratio 2.20% > threshold 0.10%
[recsys.orchestrator] === run 20261112-044228 finished: stopped_by_gate in 5.1s ===
```

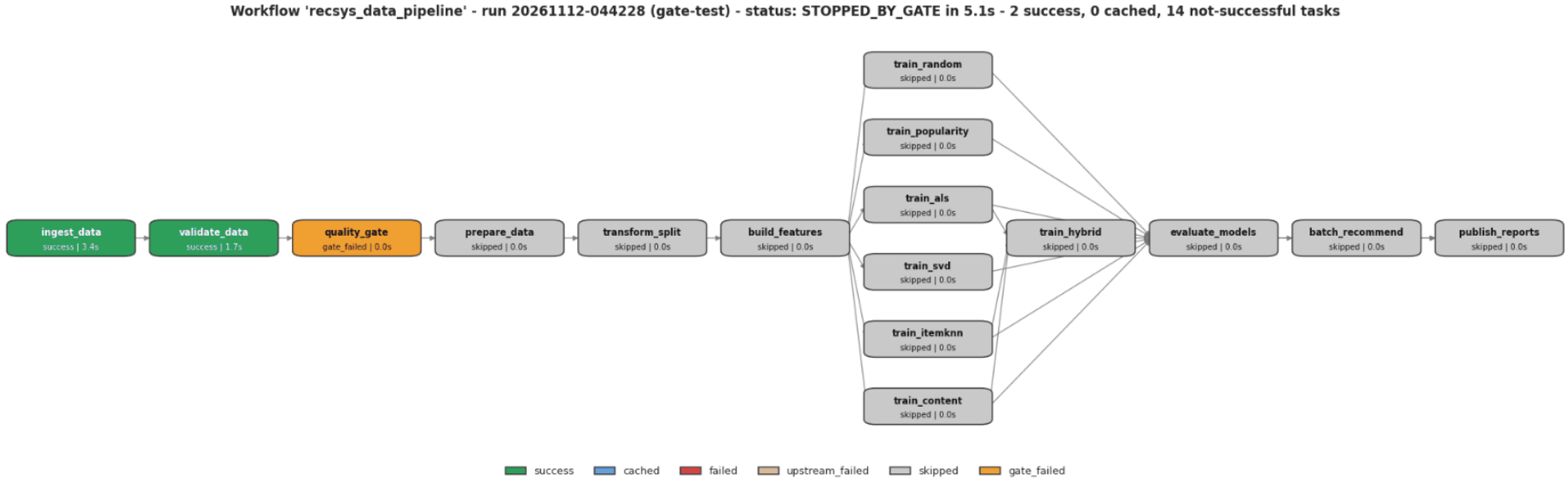


Figure 12 - Reproducibility check

Orchestrated results are identical to the stage-by-stage results (maximum difference 0.00) and the resumed run is described next.

14	batch_recommend	success	1	0.86
15	publish_reports	success	1	1.69

j.2 Reproducibility check — orchestrated run vs. stage-by-stage run

Same code, same seeds, same config → the metrics must be identical.

[49]:
```
res_orch = pd.read_csv(BASE / "reports/model_results.csv", index_col=0)
same = np.allclose(res_orch.values.astype(float), res_interactive.values.astype(float), atol=1e-9, equal_nan=True)
print(f"max |difference| over {res_orch.size} metric values: {np.nanmax(np.abs(res_orch.values.astype(float) - res_interactive.values.astype(float))):.2e}")
print("REPRODUCIBLE: orchestrated results are identical to the stage-by-stage results" if same else "WARNING: results differ - check library versions / threads")
```

```
max |difference| over 70 metric values: 0.00e+00
REPRODUCIBLE: orchestrated results are identical to the stage-by-stage results
```

j.3 Second (scheduled) run — idempotence

Re-running with `resume=True` and an unchanged configuration must not recompute anything.

[50]:
```
rec2 = wf.run(resume=True, mode="scheduled")
orch.render_run_graph(rec2, BASE / "reports/orchestration_resume_run.png"); plt.show()
print(f"{sum(t['status'] == 'cached' for t in rec2['tasks'])} of {len(rec2['tasks'])} tasks served from cache in {rec2['seconds']}s (first run: {rec['seconds']}s)")
```

```
[recsys.orchestrator] === run 20261112-044227 | workflow 'recsys_data_pipeline' | 16 tasks | resume=True ===
[recsys.orchestrator] === run 20261112-044227 finished: success in 0.0s ===
```

Workflow 'recsys_data_pipeline' - run 20261112-044227 (scheduled) - status: SUCCESS in 0.0s - 0 success, 16 cached, 0 not-successful tasks

Figure 13 - Deliverables check

Every deliverable required by the assignment is present: source modules, README, requirements, architecture diagram, validation report, feature catalog, results, MLflow store, DAG file, run screenshot and final report.

```
    ("Experiment tracking (MLflow)", "mlflow.db"), ("Orchestration DAG / workflow", "dags/recommendation_pipeline_dag.py"),
    ("Screenshot of a successful run", "reports/orchestration_run.png"), ("Batch recommendations", "reports/top10_recommendations.parquet"),
    ("Final report (2-3 pages, PDF)", "reports/final_report.pdf"), ("Run manifest", "reports/run_manifest.json"),
]
chk = pd.DataFrame([{"deliverable": d, "path": p, "status": "OK" if (BASE / p).exists() else "MISSING",
                     "size_KB": round((BASE / p).stat().st_size / 1024, 1) if (BASE / p).exists() else None} for d, p in DELIVERABLES])
display(chk.style.map(lambda v: "background-color:#c7e9c0" if v == "OK" else ("background-color:#f4b6b6" if v == "MISSING" else ""), subset=["status"]))
print("ALL DELIVERABLES PRESENT" if (chk.status == "OK").all() else "SOME DELIVERABLES ARE MISSING")
```

	deliverable	path	status	size_KB
0	Source code organised into modules	src/pipeline/tasks.py	OK	3.700000
1	README.md	README.md	OK	5.400000
2	requirements.txt	requirements.txt	OK	0.400000
3	Architecture diagram	reports/architecture_diagram.png	OK	213.300000
4	Data validation report (HTML)	reports/data_validation_report.html	OK	13.300000
5	Validation error log + error dataset	data/errors/validation_error_log.csv	OK	1105.200000
6	Feature catalog	reports/feature_catalog.csv	OK	2.700000
7	Model evaluation results	reports/model_results.csv	OK	1.400000
8	Significance tests	reports/significance_tests.csv	OK	0.900000
9	Experiment tracking (MLflow)	mlflow.db	OK	1504.000000
10	Orchestration DAG / workflow	dags/recommendation_pipeline_dag.py	OK	2.000000
11	Screenshot of a successful run	reports/orchestration_run.png	OK	183.600000
12	Batch recommendations	reports/top10_recommendations.parquet	OK	146.700000
13	Final report (2-3 pages, PDF)	reports/final_report.pdf	OK	596.500000
14	Run manifest	reports/run_manifest.json	OK	3.300000

```
ALL DELIVERABLES PRESENT
```

+ Code + Markdown

Figure 14 - Headline results and key inferences

Summary of data quality, data shape, modelling and orchestration figures from the executed run, followed by the key inferences.

```
print(f"  catalogue coverage@{K}: popularity {pop[f'Coverage@{K}']:.1%} | ALS {res_df.loc['ALS (implicit MF)', f'Coverage@{K}']:.1%} | ItemKNN {res_df.loc[knn, f'Coverage@{K}']:.1%} | Hybrid {res_df.loc[hyb, f'Cover
print(f"  SVD: RMSE {res_df.loc[svd, 'RMSE']:.4f} (popularity {pop['RMSE']:.4f}) but NDCG@{K} {res_df.loc[svd, f'NDCG@{K}']:.4f} (random {res_df.loc[res_df.index[0], f'NDCG@{K}']:.4f})")
print("\nORCHESTRATION")
print(f"  full run: {rec['status']} in {rec['seconds']}s ({len(rec['tasks'])} tasks) | cached re-run: {rec2['seconds']}s | quality-gate test: {rec3['status']} | orchestrated == stage-by-stage results: {same}")
```

```
DATA QUALITY
  users        :     20 / 4,015 rows rejected | 69 with soft warnings
  items (real) :    684 / 20,068 rows rejected | 7,110 with soft warnings
  interactions :  2,772 / 126,106 rows rejected (2.20%)
  injected fault families found: 14 of 14

DATA SHAPE
  train 98,667 | test 24,667 | cold-start test users 179 of 3,401 | matrix sparsity 99.14%

MODELLING (test window, K=10)
  best personalised model by NDCG@10: ItemKNN (cosine CF)  (0.0449 vs popularity 0.0266, +69%)
  Hybrid vs ItemKNN, paired delta NDCG@10 = -0.0007  95% CI [-0.0035, +0.0021] -> statistically tied; diversity 0.49 vs 0.40
  catalogue coverage@10: popularity 0.8% | ALS 26.4% | ItemKNN 19.1% | Hybrid 18.7%
  SVD: RMSE 0.9317 (popularity 0.9336) but NDCG@10 0.0015 (random 0.0028)

ORCHESTRATION
  full run: success in 63.4s (16 tasks) | cached re-run: 0.0s | quality-gate test: stopped_by_gate | orchestrated == stage-by-stage results: True
```

Key inferences

1. **Validation pays for itself.** About 2 % injected faults across 14 fault families (10 interaction, 4 user) were all detected by the rules, isolated in an error dataset with the violated rule, and measured against a ledger. Referential integrity *cascades*: rejecting one user (bad signup date) orphans all of that user's interactions — which is why tables are validated in dependency order. Validation also surfaced genuine problems of the *real* file (near-empty rows, duplicate book ids, missing star histograms, boilerplate genre strings).
2. **Personalisation beats popularity — and the difference is statistically meaningful.** ALS, ItemKNN and the Hybrid improve NDCG@10 over the popularity baseline with bootstrap CIs that exclude zero, while recommending far more of the catalogue. Popularity concentrates on a handful of blockbusters (several editions of the same book appear in its top list), which is exactly where a recommender adds no information.
3. **Best accuracy: neighbourhood CF; best balance: the hybrid.** ItemKNN has the highest NDCG@10 in this run, but the paired CI shows the Hybrid (ItemKNN + ALS + content + popularity) is **statistically indistinguishable** from it in NDCG, while giving **more diverse lists** and being significantly better than ALS alone. We therefore report ItemKNN as the numerically best ranker and the Hybrid as the preferable trade-off when diversity matters — we do not claim a win that the confidence interval does not support.
4. **Rating accuracy ≠ ranking quality.** SVD barely improves RMSE over the shrunk item mean and, used as a ranker, falls *below random*: ranking by predicted rating surfaces obscure, highly rated books nobody else rated (see the popularity-percentile diagnostic). Rating-prediction and top-K recommendation are different objectives and need different models and metrics.
5. **Content features alone are weak, but useful as a component.** A pure genre profile is a poor ranker (genres do not capture taste) and gives very homogeneous lists; as one signal inside the hybrid it adds only a small gain. Its real value is that it needs no other users, which matters for cold start.

Generated figure - successful run

Run graph and task timeline written by the pipeline (reports/orchestration_run.png): 16 of 16 tasks successful.

